The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
T.Siddarth Bharadwaj	**24MIP10072**
Slot	**Date of Submission**
E11	**31-03-2026**

Part A

A1 — Origin and Problem Statement

Before Git was created, developers mainly used centralized version control systems such as CVS (Concurrent Versions System) and Subversion (SVN). In these systems, all the code was stored on a central server, and developers had to connect to that server to make changes. While this approach worked for smaller projects, it created several problems when projects became large and complex.

One of the main issues was dependency on a central server. If the server went down, developers could not work on the project. This made the development process slow and unreliable. Another major problem was performance. Operations like committing changes or viewing history required communication with the server, which made them slower. Additionally, developers could not work properly without an internet connection, which limited flexibility.

These problems became more serious in large-scale projects like the Linux kernel, which is developed by thousands of contributors worldwide. Initially, the Linux project used a proprietary version control system called BitKeeper. Although BitKeeper was technically efficient and solved many performance issues, it was not open source. This created a conflict because the Linux project itself is open source, and relying on a proprietary tool went against its philosophy.

In 2005, due to licensing disagreements between the BitKeeper developers and the Linux community, access to BitKeeper was revoked. This created a critical situation because the Linux kernel development depended heavily on version control. Without a proper system, managing contributions from thousands of developers would become extremely difficult.

To solve this problem, Linus Torvalds decided to build a new version control system. His goal was not just to replace BitKeeper, but to create something better. He wanted a system that was fast, reliable, and capable of handling very large projects efficiently. Another important goal was to make it distributed, so that every developer could have a complete copy of the repository.

As a result, Git was developed in a very short period of time. One of its key features is the distributed model. Unlike centralized systems, Git allows every developer to have a full copy of the project, including its history. This means developers can work offline and still perform operations like commits, branching, and merging.

Git was also designed with a strong focus on performance. Most operations are done locally, which makes them extremely fast. It also uses a unique way of storing data, where each change is tracked efficiently, ensuring data integrity.

Another important feature of Git is its powerful branching and merging capabilities. Developers can create branches easily and work on different features without affecting the main codebase. Later, these changes can be merged smoothly. This made collaboration much easier compared to older systems.

Over time, Git became widely adopted not only for the Linux kernel but also for almost all types of software development. Platforms like GitHub, GitLab, and Bitbucket are built around Git and provide additional features like code hosting, collaboration, and version tracking.

From my understanding, Git solved both technical and philosophical problems. It removed dependency on proprietary tools and gave developers full control over their code. It also improved speed, reliability, and collaboration in software development.

Today, Git is considered an essential tool in modern development and is used by individuals, startups, and large companies around the world.

A2 — License Analysis (GPL v2)

Git is released under the GNU General Public License version 2 (GPL v2), which is one of the most important licenses in the open-source ecosystem. This license is designed to ensure that software remains free in terms of user freedom and not just cost.

The GPL license is based on the concept of four fundamental freedoms. These freedoms define what users are allowed to do with the software. The first freedom is the ability to run the program for any purpose, without any restrictions. The second freedom is the ability to study how the program works, which is possible because the source code is openly available. The third freedom is the ability to modify the program according to one's needs. The fourth freedom is the ability to distribute copies of the software, including modified versions.

Git provides all these freedoms because its source code is publicly available and can be accessed by anyone. This means that developers can not only use Git but also understand its internal working and improve it.

One of the key features of the GPL license is that it is a "copyleft" license. This means that if someone modifies Git and distributes the modified version, they are required to release their changes under the same GPL license. This ensures that improvements made to the software remain open and available to the community. In other words, it prevents companies from taking open-source software, modifying it, and turning it into completely closed, proprietary software.

This also answers an important question: can a company take Git, modify it, and sell it without sharing their changes? The answer is no. While a company can sell Git or services related to it, if they distribute a modified version, they must also provide the source code of those modifications under the GPL license.

To better understand this, it is useful to compare GPL with a permissive license like the MIT License. The MIT License allows anyone to use, modify, and distribute software with very few restrictions. A company can take MIT-licensed software, modify it, and keep the changes private. In contrast, GPL enforces openness by requiring that all distributed modifications remain open source.

Feature	GPL License	MIT License
Sharing modifications	Mandatory	Not required
Commercial use	Allowed, but must share changes	Fully allowed without sharing
Philosophy	Protect user freedom	Maximize flexibility

From my perspective, both licenses have their own advantages. GPL is suitable when the goal is to ensure that software always remains open and benefits the community. On the other hand, MIT is useful when the goal is wider adoption, especially in commercial environments.

Another important concept is the difference between “free as in free beer” and “free as in freedom.” “Free beer” means something is available at no cost, while “free as in freedom” means users have control over the software. Git represents “free as in freedom” because users are not restricted in how they use or modify it.

In my opinion, the GPL license plays a major role in maintaining the open nature of Git. It ensures that the software cannot become closed over time and that all improvements contribute back to the community. This makes Git not just a tool, but a part of a larger open-source movement.

A3 — Ethics of Open Source

Open source is not just a technical concept or a licensing model; it represents a broader philosophy about how knowledge and tools should be shared. It raises important ethical questions about ownership, collaboration, and fairness in software development.

One major question is whether all software should be open source. In my opinion, not all software needs to be open source. Companies invest significant time, effort, and money into developing their products, and they should have the right to protect their work and generate profit. Proprietary software can also provide structured support, security updates, and stable services, which are important in many real-world applications. However, open source should be encouraged in areas where transparency and accessibility are important, such as education, research, and security-related tools.

Another important ethical issue is whether it is fair for companies to build profitable products using open-source software. Many large companies like Google, Red Hat, and Meta use open-source technologies such as Linux. On one hand, this can be seen as positive because these companies often contribute back to the community by improving the software and funding development. On the other hand, it can also be argued that they benefit from the free work of open-source contributors while generating large profits.

In my view, this situation is not completely unfair as long as companies contribute back in some way. Open source is based on voluntary contribution, and its goal is to promote collaboration. If companies use open-source software responsibly and support the ecosystem, it creates a balance between profit and community benefit.

The phrase “standing on the shoulders of giants” is very relevant in the context of open source. It means that new innovations are built on existing knowledge. Open-source software makes this possible because developers can study existing code, learn from it, and improve upon it. This accelerates innovation because developers do not have to start from scratch every time.

At the same time, there is a concern that relying too much on open-source code may reduce originality. Some developers may simply reuse existing solutions instead of creating new ones. However, I believe that open source does not reduce innovation; instead, it shifts the focus from reinventing basic tools to solving more complex problems.

From my perspective, open source creates a more collaborative and inclusive environment in software development. It allows students, independent developers, and professionals to contribute and learn from each other. While there are some ethical concerns regarding commercial use, the overall impact of open source is positive.

In conclusion, open source should not completely replace proprietary software, but it should continue to grow alongside it. A balanced approach, where both models coexist, is the most practical and beneficial for the future of technology.

Part B

Linux Footprint of Git

Git is a command-line tool and does not run as a background service like a web server. Instead, it is executed only when the user runs a Git command.

Installation Method

On my system (Pop!_OS 24.04), Git is installed using the APT package manager. The installation command is:

```
sudo apt install git
```

This method allows easy installation and automatic updates through the system repositories.

File Locations and Directories

After installation, Git is stored in standard Linux directories.

The main executable file is located at:

```
/usr/bin/git
```

This directory contains essential system binaries that are accessible to all users.

Git configuration is stored in the user's home directory:

```
~/.gitconfig
```

This file contains user-specific settings such as username and email configuration.

User and Permissions

Git does not run under a separate system user or service account. It runs under the currently logged-in user.

For example, on my system, Git runs under my user account when I execute commands. This means that permissions depend on the user's access rights.

This design improves flexibility, as each user can have their own Git configuration and repositories.

Service Management

Git is not a daemon or background service, so it is not managed using systemctl. It does not need to be started or stopped like services such as Apache or MySQL.

Instead, Git is executed directly from the terminal when required.

Update Model

Git updates are managed through the Linux package manager.

The open-source community maintains Git and releases updates regularly. These updates are distributed through package repositories.

Users can update Git using the following commands:

```
sudo apt update
sudo apt upgrade git
```

This ensures that the system receives the latest features and security patches.

From my observation, Git is lightweight and does not require background execution, which makes it efficient and easy to use compared to server-based software.

ScreenShots

```
~/Programming /OSS_Vityatri_Project
❯ which git
/usr/bin/git
```

```
~/Programming /OSS_Vityatri_Project
❯ git --version
git version 2.43.0
```

```
~/Programming /OSS_Vityatri_Project
❯ ls -l /usr/bin/git
-rwxr-xr-x 1 root root 4066232 Jul  3  2025 /usr/bin/git
```

Part C

The FOSS Ecosystem Around Git

Git does not function as an isolated tool; it is part of a much larger open-source ecosystem that supports modern software development. Its true strength lies not only in its core functionality but also in how it integrates with other tools, platforms, and communities.

Dependencies and Supporting Technologies

Git is primarily written in the C programming language and relies on standard system libraries available in Linux environments. It also uses supporting technologies such as SSH (Secure Shell) for secure communication between systems and OpenSSL for encryption. These components ensure that data transfer between repositories is secure and reliable.

In addition, Git works closely with the Linux command-line environment (Bash), which allows users to automate tasks and integrate Git into scripts. This combination of Git and shell scripting creates a powerful workflow for developers.

Git is also commonly used alongside other open-source tools, such as:

- Jenkins for continuous integration and deployment
- Docker for containerization
- Build tools like Make and Maven

These tools together form a complete development pipeline, where Git manages the source code while other tools handle testing, building, and deployment.

Platforms and Tools Built Around Git

One of the biggest impacts of Git is the ecosystem of platforms built around it. Git itself is only a version control system, but platforms like GitHub, GitLab, and Bitbucket provide additional features such as remote repository hosting, issue tracking, pull requests, and team collaboration.

GitHub, for example, has become one of the most popular platforms for hosting open-source projects. It allows developers from different parts of the world to collaborate on the same project. Similarly, GitLab provides integrated DevOps tools, and Bitbucket offers enterprise-focused solutions.

These platforms have made Git more accessible and have significantly improved collaboration in software development.

What Git Has Enabled

Git has enabled the concept of distributed development, where multiple developers can work on the same project independently. Each developer has a complete copy of the repository, which allows them to experiment, create branches, and test changes without affecting others.

It has also made open-source collaboration easier. Developers can contribute to projects by forking repositories, making changes, and submitting pull requests. This process has become a standard workflow in the software industry.

Git has also influenced the creation of many tools and practices, such as:

- Code review systems
- Continuous integration pipelines
- Agile and collaborative workflows

These practices have improved the quality and speed of software development.

Role in the Development Stack

Although Git is not directly part of the LAMP stack (Linux, Apache, MySQL, PHP), it plays an important supporting role. Developers use Git to manage the source code of applications that run on these technologies.

For example, a web application built using the LAMP stack will have its source code managed using Git. Developers can track changes, revert to previous versions, and collaborate efficiently using Git.

This makes Git an essential tool in almost every software development environment, even if it is not directly part of the runtime stack.

Community and Governance

Git is maintained by a global open-source community, with Linus Torvalds as its original creator. Contributions come from developers around the world, who help improve the software by fixing bugs, adding features, and maintaining documentation.

The development process is transparent and community-driven. Discussions take place through mailing lists, forums, and online repositories. This open model ensures that decisions are not controlled by a single company, but are influenced by the needs of the community.

The strong community support also makes it easier for beginners to learn Git. There are many tutorials, forums, and resources available online.

Real-World Usage and Impact

Git is widely used in almost all areas of software development. It is used by:

- Students for academic projects
- Startups for building products
- Large companies for managing complex systems

Companies like Google, Microsoft, and Meta use Git or Git-based workflows in their development processes. This shows how important Git has become in the industry.

From my observation, Git is not just a tool but a foundation for modern development practices. It connects developers, tools, and platforms into a unified ecosystem.

Part D

Open Source vs Proprietary (Git)

Dimensions	Git (OpenSource)	Proprietary Alternative (e.g., Perforce)
Cost	Free to use and distribute	Requires paid license
Security	Source code is open and can be audited	Source code is closed, must trust vendor
Support & Reliability	Community support, forums, documentation	Official company support and SLAs
Freedom to Modify	Full freedom to modify and share	No access to modify source code
Control	Community-driven development	Controlled by a company
Performance	Fast and distributed (works offline)	Often centralized, depends on server

Conclusion

After comparing Git with proprietary alternatives such as Perforce, it becomes clear that both open-source and proprietary models have their own advantages and limitations. Git, being an open-source tool, provides a high level of flexibility and freedom to users. It allows developers to access the source code, modify it according to their needs, and share their improvements with others. This level of openness encourages learning, experimentation, and collaboration, which are essential in modern software development.

One of the most significant advantages of Git is its distributed nature. Unlike many proprietary systems that depend on a central server, Git allows every developer to have a complete copy of the repository. This improves reliability and allows work to continue even without an internet connection. From my experience while using Git during this project, I found it to be fast, efficient, and easy to use once the basic commands are understood. It also integrates well with platforms like GitHub, making collaboration smoother.

However, proprietary tools also have their own strengths. They often provide dedicated customer support, structured environments, and enterprise-level features that may not always be available in open-source tools. For large organizations that require strict control, compliance, and guaranteed support, proprietary solutions can sometimes be more suitable. This shows that open source is not always a complete replacement for proprietary software, but rather an alternative with different benefits.

In my opinion, Git is highly suitable for real-world use, especially for students, individual developers, and startups. It is lightweight, widely adopted, and supported by a large community. The availability of tutorials, forums, and documentation makes it easier to learn and troubleshoot problems. During this project, I also realized how important Git is in almost every development workflow, from small personal projects to large-scale applications.

Another important aspect is the long-term impact of open-source tools like Git. They reduce dependency on specific companies and give users more control over their work. This aligns with the idea of “freedom” in open source, where users are not locked into a particular platform or vendor. At the same time, the existence of proprietary tools ensures competition and continuous improvement in the industry.

Overall, I would strongly recommend Git for most development scenarios. It offers a good balance of performance, flexibility, and community support. I would also be interested in contributing to open-source projects in the future, as it provides an opportunity to learn from real-world code and collaborate with developers globally.

In conclusion, both open-source and proprietary software have their place in the technology ecosystem. Instead of choosing one over the other, it is more practical to understand their strengths and use them based on the requirements. This project helped me develop a better understanding of this balance and the role of open-source software in shaping modern technology.